

Universidad Nacional Autónoma de México



Facultad de Ingeniería

División de Ingeniería Eléctrica



Laboratorio Computación Gráfica e Interacción
Humano-Computadora (6590)

Práctica 2: Proyecciones y puertos de vista.

Transformaciones Geométricas

Nombre: Calderón Gutiérrez Victor Emiliano

No. Cuenta: 423097331

Grupo (laboratorio): 03

Grupo (teoría): 02

Semestre 2026-1

Fecha de entrega: 31/08/2025

Calificación: _____

1.Ejercicios

1.1. Ejercicio 1

Para realizar esta parte de la actividad primer se comenzó por la creación de los vectores para dibujar cada una de las letras. Cómo se vio en el ejercicio de clase, es posible crear varios vectores con cada uno de los elementos a crear y después renderizarlos por separado haciendo uso del *meshColorList*, por lo que se generaron 3 vectores diferentes para cada una de mis iniciales, además, en cada uno de ellos se generó la parte RGB con algunos de mis colores favoritos.

Para la letra E:

```
98 // Letra E
99 GLfloat vertices_letraE[] = {
100     -0.9f,  0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
101     -0.8f,  0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
102     -0.9f, -0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
103
104     -0.9f, -0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
105     -0.8f,  0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
106     -0.8f, -0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
107
108     -0.8f,  0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
109     -0.4f,  0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
110     -0.4f,  0.4f,  0.0f,      0.424f,  0.275f,  0.459f,
111
112     -0.8f,  0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
113     -0.4f,  0.4f,  0.0f,      0.424f,  0.275f,  0.459f,
114     -0.8f,  0.4f,  0.0f,      0.424f,  0.275f,  0.459f,
115
116     -0.8f, -0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
117     -0.4f, -0.4f,  0.0f,      0.424f,  0.275f,  0.459f,
118     -0.8f, -0.4f,  0.0f,      0.424f,  0.275f,  0.459f,
119
120     -0.8f, -0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
121     -0.4f, -0.5f,  0.0f,      0.424f,  0.275f,  0.459f,
122     -0.4f, -0.4f,  0.0f,      0.424f,  0.275f,  0.459f,
123
124     -0.8f,  0.05f,  0.0f,      0.424f,  0.275f,  0.459f,
125     -0.4f,  0.05f,  0.0f,      0.424f,  0.275f,  0.459f,
126     -0.8f, -0.05f,  0.0f,      0.424f,  0.275f,  0.459f,
127
128     -0.8f, -0.05f,  0.0f,      0.424f,  0.275f,  0.459f,
129     -0.4f,  0.05f,  0.0f,      0.424f,  0.275f,  0.459f,
130     -0.4f, -0.05f,  0.0f,      0.424f,  0.275f,  0.459f,
131 },
132
```

Para la letra: C:

```
133 // Letra C
134 GLfloat vertices_letraC[] = {
135     -0.3f, 0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
136     -0.2f, 0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
137     -0.3f, -0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
138
139     -0.3f, -0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
140     -0.2f, 0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
141     -0.2f, -0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
142
143     -0.2f, 0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
144     0.3f, 0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
145     0.3f, 0.4f, 0.0f, 0.259f, 0.478f, 0.255f,
146
147     -0.2f, 0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
148     0.3f, 0.4f, 0.0f, 0.259f, 0.478f, 0.255f,
149     -0.2f, 0.4f, 0.0f, 0.259f, 0.478f, 0.255f,
150
151     -0.2f, -0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
152     0.3f, -0.4f, 0.0f, 0.259f, 0.478f, 0.255f,
153     -0.2f, -0.4f, 0.0f, 0.259f, 0.478f, 0.255f,
154
155     -0.2f, -0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
156     0.3f, -0.5f, 0.0f, 0.259f, 0.478f, 0.255f,
157     0.3f, -0.4f, 0.0f, 0.259f, 0.478f, 0.255f,
158 };
159
```

Para la letra G:

```
160 // Letra G
161 GLfloat vertices_letraG[] = {
162     0.5f, 0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
163     0.4f, 0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
164     0.5f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
165
166     0.5f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
167     0.4f, 0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
168     0.4f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
169
170     0.5f, 0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
171     0.9f, 0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
172     0.9f, 0.4f, 0.0f, 0.961f, 0.251f, 0.129f,
173
174     0.5f, 0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
175     0.9f, 0.4f, 0.0f, 0.961f, 0.251f, 0.129f,
176     0.5f, 0.4f, 0.0f, 0.961f, 0.251f, 0.129f,
177
178     0.5f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
179     0.9f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
180     0.9f, -0.4f, 0.0f, 0.961f, 0.251f, 0.129f,
181
182     0.5f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
183     0.9f, -0.4f, 0.0f, 0.961f, 0.251f, 0.129f,
184     0.5f, -0.4f, 0.0f, 0.961f, 0.251f, 0.129f,
185
186     0.7f, 0.05f, 0.0f, 0.961f, 0.251f, 0.129f,
187     0.9f, 0.05f, 0.0f, 0.961f, 0.251f, 0.129f,
188     0.7f, -0.05f, 0.0f, 0.961f, 0.251f, 0.129f,
189
190     0.7f, -0.05f, 0.0f, 0.961f, 0.251f, 0.129f,
191     0.9f, 0.05f, 0.0f, 0.961f, 0.251f, 0.129f,
192     0.9f, -0.05f, 0.0f, 0.961f, 0.251f, 0.129f,
193
194     0.9f, 0.0f, 0.0f, 0.961f, 0.251f, 0.129f,
195     0.8f, 0.0f, 0.0f, 0.961f, 0.251f, 0.129f,
196     0.9f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
197
198     0.9f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
199     0.8f, 0.0f, 0.0f, 0.961f, 0.251f, 0.129f,
200     0.8f, -0.5f, 0.0f, 0.961f, 0.251f, 0.129f,
201 };
202
```

Una vez que se generaron los vectores, se generaron los mesh, los que nos permiten dar color a cada una de las letras.

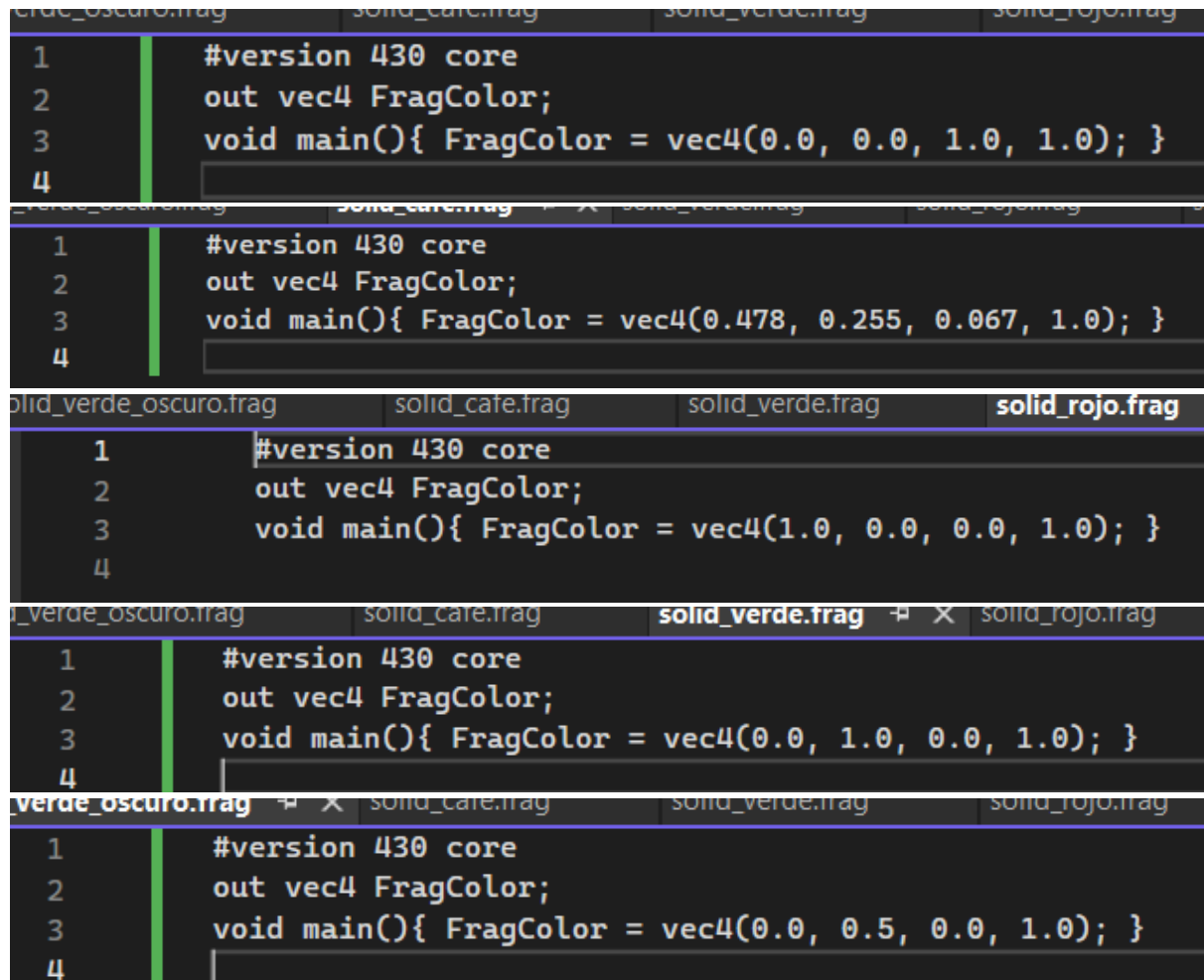
```
257
258 //Para las letras hay que usar el segundo set de shaders con índice 1 en ShaderList
259 shaderList[1].useShader();
260 uniformModel = shaderList[1].getModelLocation();
261 uniformProjection = shaderList[1].getProjectLocation();
262
263 // Bucle para dibujar cada letra
264 for (int i = 0; i < meshColorList.size(); i++)
265 {
266     model = glm::mat4(1.0);
267     model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));
268     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
269     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
270     meshColorList[i]->RenderMeshColor();
271 }
272
273 glUseProgram(0);
274 mainWindow.swapBuffers();
275 }
276 return 0;
277 }
278
```

En este bloque de código primero se seleccionó el shader en índice 1 del *shaderList* esto debido a que el código base venía definido para la parte de los colores en las letras esa parte del arreglo. Además, para poder dibujar cada una de las letras (de acuerdo con el ejercicio de clase y el código base) siempre se hace uso el mismo código, por lo que se metió a un *for* para lograr recorrer el *meshColorList* para todos los colores para cada letra, de esta forma se puede automatizar en caso de haber más colores y figuras. El resultado se muestra a continuación:



1.2. Ejercicio 2

Para esta otra actividad se tuvo que recrear la figura que se hizo en el ejercicio de clase, que fue la casa. La realización de este fue más compleja, ya que primero se tuvieron que crear archivos nuevos que contenían los colores a usar:



```
1 #version 430 core
2 out vec4 FragColor;
3 void main(){ FragColor = vec4(0.0, 0.0, 1.0, 1.0); }
4

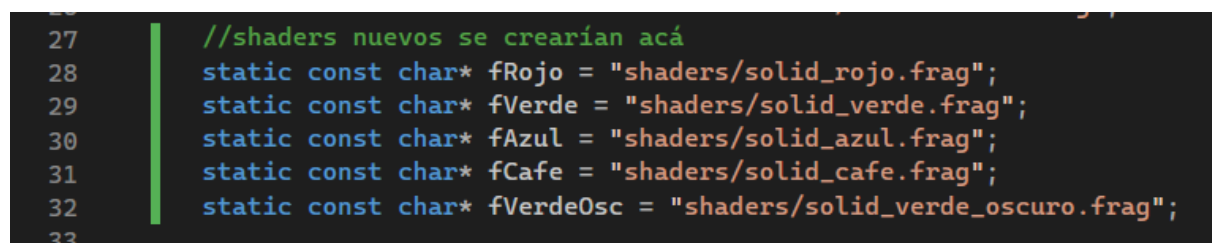
1 #version 430 core
2 out vec4 FragColor;
3 void main(){ FragColor = vec4(0.478, 0.255, 0.067, 1.0); }
4

1 #version 430 core
2 out vec4 FragColor;
3 void main(){ FragColor = vec4(1.0, 0.0, 0.0, 1.0); }
4

1 #version 430 core
2 out vec4 FragColor;
3 void main(){ FragColor = vec4(0.0, 1.0, 0.0, 1.0); }
4

1 #version 430 core
2 out vec4 FragColor;
3 void main(){ FragColor = vec4(0.0, 0.5, 0.0, 1.0); }
4
```

Todos estos fueron para cada uno de los colores y debido a que son archivos externos se tuvo que también generar el header necesario para poder usarlo en el archivo main.



```
27 //shaders nuevos se crearían acá
28 static const char* fRojo = "shaders/solid_rojo.frag";
29 static const char* fVerde = "shaders/solid_verde.frag";
30 static const char* fAzul = "shaders/solid_azul.frag";
31 static const char* fCafe = "shaders/solid_cafe.frag";
32 static const char* fVerdeOsc = "shaders/solid_verde_oscuro.frag";
33
```

Una vez que estos shaders estaban listos para ser usados se inicializaron para que se pudieran usar en el main. Para esto se hizo uso del *shaderList*.

```
222 void CreateShaders()
223 {
224     shaderList.clear();
225
226     auto mk = [&](const char* vs, const char* fs) {
227         Shader* s = new Shader();
228         s->CreateFromFiles(vs, fs);
229         shaderList.push_back(*s);
230     };
231     mk(vShader, fRojo);           // 0
232     mk(vShader, fVerde);          // 1
233     mk(vShader, fAzul);           // 2
234     mk(vShader, fCafe);           // 3
235     mk(vShader, fVerde0sc);       // 4
236 }
```

Para importante, se comentó el código que se tenía para las letras, esto para que no interfiriera en la presentación, así también para mantener el registro de ambas actividades.

```
/*
//Para las letras hay que usar el segundo set de shaders con índice 1 en ShaderList
shaderList[1].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

// Bucle para dibujar cada letra
for (int i = 0; i < meshColorList.size(); i++)
{
    model = glm::mat4(1.0);
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
    meshColorList[i]->RenderMeshColor();
}
*/
```

Posterior a eso se inició con la creación de cada parte de la casa.

```
283 // Techo
284 shaderList[2].useShader();
285 GLuint uM = shaderList[2].getModelLocation();
286 GLuint uP = shaderList[2].getProjectLocation();
287 glUniformMatrix4fv(uP, 1, GL_FALSE, glm::value_ptr(projection));
288 glm::mat4 model(1.0f);
289 model = glm::translate(model, glm::vec3(0.0f, 0.6f, -2.0f));
290 model = glm::scale(model, glm::vec3(1.30f, 0.70f, 1.0f));
291 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
292 meshList[0]->RenderMesh();
293
294 // Cubo rojo
295 shaderList[0].useShader();
296 uM = shaderList[0].getModelLocation(); uP = shaderList[0].getProjectLocation();
297 glUniformMatrix4fv(uP, 1, GL_FALSE, glm::value_ptr(projection));
298 model = glm::mat4(1.0f);
299 model = glm::translate(model, glm::vec3(0.0f, -0.25f, -2.05f));
300 model = glm::scale(model, glm::vec3(1.12f, 1.02f, 1.0f));
301 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
302 meshList[1]->RenderMesh();
```

```
304 // Ventana izquierda
305 shaderList[1].useShader();
306 uM = shaderList[1].getModelLocation(); uP = shaderList[1].getProjectLocation();
307 glUniformMatrix4fv(uP, 1, GL_FALSE, glm::value_ptr(projection));
308 model = glm::mat4(1.0f);
309 model = glm::translate(model, glm::vec3(-0.38f, 0.06f, 0.0f));
310 model = glm::scale(model, glm::vec3(0.30f, 0.28f, 1.0f));
311 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
312 meshList[1]->RenderMesh();
313
314 // Ventana derecha
315 model = glm::mat4(1.0f);
316 model = glm::translate(model, glm::vec3(0.38f, 0.06f, -1.95f));
317 model = glm::scale(model, glm::vec3(0.30f, 0.28f, 1.0f));
318 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
319 meshList[1]->RenderMesh();
320
321 // Puerta
322 model = glm::mat4(1.0f);
323 model = glm::translate(model, glm::vec3(0.0f, -0.45f, -1.95f));
324 model = glm::scale(model, glm::vec3(0.30f, 0.42f, 1.0f));
325 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
326 meshList[1]->RenderMesh();
```

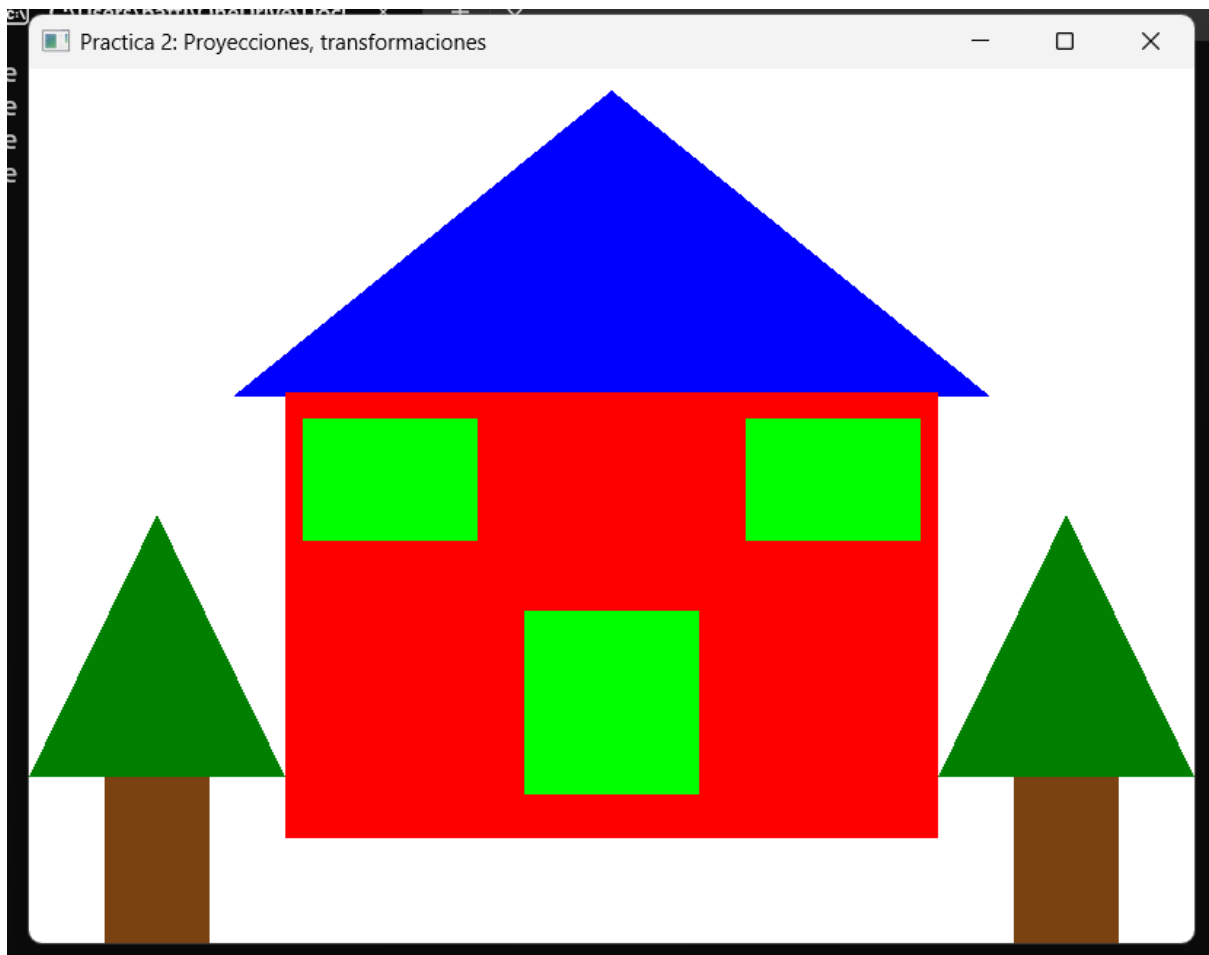
```
328 // Pino izquierdo
329 shaderList[4].useShader();
330 uM = shaderList[4].getModelLocation(); uP = shaderList[4].getProjectLocation();
331 glUniformMatrix4fv(uP, 1, GL_FALSE, glm::value_ptr(projection));
332 model = glm::mat4(1.0f);
333 model = glm::translate(model, glm::vec3(-0.78f, -0.32f, -2.0f));
334 model = glm::scale(model, glm::vec3(0.44f, 0.6f, 1.0f));
335 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
336 meshList[0]->RenderMesh();
337
338 // Tronco izquierdo
339 shaderList[3].useShader();
340 uM = shaderList[3].getModelLocation(); uP = shaderList[3].getProjectLocation();
341 glUniformMatrix4fv(uP, 1, GL_FALSE, glm::value_ptr(projection));
342 model = glm::mat4(1.0f);
343 model = glm::translate(model, glm::vec3(-0.78f, -0.82f, -1.9f));
344 model = glm::scale(model, glm::vec3(0.18f, 0.4f, 1.0f));
345 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
346 meshList[1]->RenderMesh();
```

```

348 // Pino derecho
349 shaderList[4].useShader();
350 uM = shaderList[4].getModelLocation(); uP = shaderList[4].getProjectLocation();
351 glUniformMatrix4fv(uP, 1, GL_FALSE, glm::value_ptr(projection));
352 model = glm::mat4(1.0f);
353 model = glm::translate(model, glm::vec3(0.78f, -0.32f, -2.0f));
354 model = glm::scale(model, glm::vec3(0.44f, 0.6f, 1.0f));
355 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
356 meshList[0]->RenderMesh();
357
358 // Tronco derecho
359 shaderList[3].useShader();
360 uM = shaderList[3].getModelLocation(); uP = shaderList[3].getProjectLocation();
361 glUniformMatrix4fv(uP, 1, GL_FALSE, glm::value_ptr(projection));
362 model = glm::mat4(1.0f);
363 model = glm::translate(model, glm::vec3(0.78f, -0.82f, -1.9f));
364 model = glm::scale(model, glm::vec3(0.18f, 0.4f, 1.0f));
365 glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
366 meshList[1]->RenderMesh();

```

Y al compilar y correr el programa se puede observar la casa.



2. Problemas

Creo que el mayor problema que se llegó a presentar durante la realización de la práctica fue la capacidad de imaginarme en el espacio en el que estaba programando, esto debido a que ya estamos tratando con espacios tridimensionales y aunque a veces a prueba y error puede

salir, creo que es importante saber que pueden existir recursos matemáticos que me podrían ahorrar horas de trabajo.

3. Conclusión

La práctica fue bastante retadora y me permitió entender de gran manera cual es el pipeline para poder generar gráficos usando OpenGL. Esto es algo que hemos visto desde la práctica 1, pero ahora se pudo ver en algo más complejo y completo, en un entorno 3d, en donde ahora si se pudo aplicar figuras que necesitan de un espacio tridimensional. Creo que esta práctica tuvo el nivel suficiente para mantenerme por horas estudiando y entendiendo que es lo que hacía cada comando, así como también mucha prueba y error para poder determinar cuales serían los valores “adecuados” para poder crear tanto las figuras, los colores, los shaders y muchos elementos que se vieron en toda la realización. En general, considero que aprendí suficiente y me deja bien parado para poder entender nuevos conceptos que implementen cosas más complejas con respecto a lo visto en esta práctica.

4. Bibliografía

- *#427a41 código de color hex.* (s. f.). Encycolorpedia. Recuperado 31 de agosto de 2025, de <https://encycolorpedia.es/427a41>
- *Naranja tráfico.* (s. f.). Encycolorpedia. Recuperado 31 de agosto de 2025, de <https://encycolorpedia.es/f54021>
- *Lila azulado.* (s. f.). Encycolorpedia. Recuperado 31 de agosto de 2025, de <https://encycolorpedia.es/6c4675>